The server-side model has many advantages, but ten years ago, it was pretty limited in terms of user interaction. This created problems when users began to expect web applications to have the same rich and complex interactions as desktop interactions. Client-side logic was one response to this problem. Another was making the browser markup, particularly CSS, more powerful to allow browsers more access to complex interactivity.

Designing Around Client Logic

As a web application begins to act more like a desktop application, the application needs to maintain a lot of state and logic that only pertains to the user interface (UI). It doesn't always make sense to manage the client-only information and logic on the server, so JavaScript applications in the browser became more and more complex.

The interactions a user has that are managed by JavaScript may be harder to model as CRUD resources and actions. SPA JavaScript frameworks often have a different set of actions. As a result, these frameworks have structured themselves quite differently from server-side Rails applications. For example, a primary concern of JavaScript application frameworks is managing the state of the objects and data being interacted with on the client (for example, which items are active), and the frameworks often emphasize having a lot of relatively small constructs that manage the data and logic for a small part of the page. Therefore, a problem in a lot of client-side app frameworks is sharing common state information among otherwise unrelated small components across the page or the app.

On the server side, sharing common state is not a concern in the same way. A server-side Rails app stores global state in the database and generally doesn't worry about the mutability of individual instances because they don't last beyond a single request. How, then, would we structure an application that combines a lot of server-side logic that is Rails-shaped with a lot of client-side interaction that is JavaScript-shaped?

One option is to do as little in JavaScript as possible. In this paradigm, the server